



Département mathématiques et informatique
UFR des Sciences

**Licence informatique - 2ème année
(YSINL4B1) Programmation Fonctionnelle
1 heure et 30 minutes**

Vous devez remettre ce sujet à votre surveillant à la fin du TP noté.

Les notes prises pendant les séances CM et TP en version papier sont autorisées, ainsi que les TP préparés et le support du cours dans vos répertoires personnels sur votre compte informatique de l'université de Caen Normandie ou sur la page ecampus du cours¹. L'accès à internet, les ordinateurs portables, téléphones portables, montres, clés usb et plus généralement tout matériel électronique sont interdits.

Le TP noté est à faire **individuellement** et **exclusivement sur une machine de l'université de Caen Normandie**. Toute communication avec une autre personne ou avec une intelligence artificielle est interdite et sera passible d'un conseil de discipline. Les sites consultés sur internet sont enregistrés pour chaque session pendant toute la durée du TP noté et feront l'objet d'une vérification.

Consignes :

- Créez un fichier texte `NomPrenom_NumEtu_GroupeTD.hs` où `NomPrenom` est votre nom et votre prénom, `NumEtu` est votre numéro d'étudiant et `GroupeTD` est votre groupe TD.
- Écrivez (en commentaire) au début du fichier que vous rendez : votre nom, votre prénom, votre numéro d'étudiant et votre groupe TD.
- Écrivez dans le fichier que vous rendez vos réponses aux questions du TP *en indiquant à chaque fois le numéro de la question traitée*.
- A la fin du TP, téléversez (i.e. upload) votre fichier sous ecampus au dépôt **Dépôt du TP noté INFO** disponible en haut de la page du cours sur ecampus :
 - cet envoi doit être fait impérativement AVANT 17H30
 - conservez une COPIE de votre fichier et vérifiez bien que le fichier envoyé est le BON.
 - les envois par mail ne sont pas acceptés.

Toutes les fonctions définies **doivent être signées**.

Exercice 1 : (6 points)

1. Définir une fonction `premNegatif` qui prend en paramètre une liste `xs` d'entiers, et retourne le premier élément négatif de la liste s'il en existe, sinon la fonction retourne 0. La définition doit utiliser soit une notation par garde soit une expression conditionnelle `if-then-else`.

Exemple : `premNegatif [1,2] ==> 0`, `premNegatif [5,6,-7,8,-9] ==> -7`

1. <https://ecampus.unicaen.fr/course/view.php?id=53232>

```

-- réponse
premNegatif, premNeg :: [Int] -> Int
premNegatif [] = 0
premNegatif (x:xs)
  | x < 0 = x
  | otherwise = premNegatif xs

premNeg [] = 0
premNeg (x:xs) = if x < 0 then x else premNeg xs

```

2. Définir une fonction `decaler` qui prend en paramètre une liste d'entiers `xs`, déplace son premier élément à la dernière place et retourne la liste ainsi modifiée. Si la liste `xs` est vide, la fonction ne fera aucun décalage. Votre définition doit utiliser le filtrage par motifs (pattern matching) sans utiliser des fonctions standards d'Haskell.

Exemple : `decaler [10,2,3,4,5,9,10] ==> [2,3,4,5,9,10,10]`, `decaler [2] ==> [2]`

```

-- réponse
decaler :: [Int] -> [Int]
decaler [] = []
decaler (x:xs) = xs ++ [x]

```

3. Définir à l'aide d'une ZF-expression une fonction `listMultiple5` qui prend en paramètre une liste `xs` d'entiers et retourne les éléments de `xs` qui sont des multiples du nombre 5 (peuvent être divisés par 5). Vous pouvez utiliser une fonction standard d'Haskell pour définir `listMultiple5`.

Exemple : `listMultiple5 [10,2,30,4,15,7,9,10] ==> [10,30,15,10]`
`listMultiple5 [12,2,3,4,11,7,9] ==> []`

```

-- réponse
listMultiple5 :: [Int] -> [Int]
listMultiple5 xs = [x | x<-xs , mod x 5 == 0 ]

```

4. Définir une fonction `tailles` qui prend en paramètre une liste de chaînes de caractères et retourne la liste de leurs longueurs. Votre définition doit obligatoirement utiliser une fonction d'ordre supérieur telle que `filter`, `map`,... Par ailleurs, vous pouvez utiliser en plus une fonction standard d'Haskell pour définir `tailles`.

Exemple : `tailles ["je" , "mange" , "une" , "pomme"] ==> [2,5,3,5]`

```

-- réponse
tailles :: [String] -> [Int]
tailles xs = map length xs

```

Exercice 2 : (7 points)

1. Définir une fonction `separerMoy` qui prend en paramètre une liste d'entiers, sépare cette liste en deux sous-listes selon la valeur moyenne de la liste (la première liste comprenant les entiers inférieurs ou égaux et la deuxième liste comprenant les entiers supérieurs à cette moyenne) et retourne le couple formé de ces deux sous-listes. Pour calculer la moyenne de la liste, on pourra prendre la division entière de la somme

de ces valeurs par sa longueur. Vous pouvez vous servir des fonctions standards simples pour définir `separerMoy`.

Exemple : `separerMoy [3,7,1,5,3,8,4] ==> ([3,1,3,4],[7,5,8]) -- moy = 4 (= div 31 7 )`

```
-- réponse
separerMoy :: [Int] -> ([Int],[Int])
separerMoy ls = ([e1 | e1 <- ls, e1 <= moyenne], [e1 | e1 <- ls, e1 > moyenne])
  where moyenne = div (sum ls) (length ls)
```

2. On envisage la fonction `sommeCarre` suivante qui prend en entrée une liste d'entiers et qui retourne la somme de ces entiers élevés au carré :

```
sommeCarre :: [Integer] -> Integer
sommeCarre [] = 0
sommeCarre (x:xs) = x*x + sommeCarre xs

-->sommeCarre [1,3,5] ==> 35
```

Cette fonction est récursive non-terminale. Redéfinir cette fonction en une fonction terminale `sommeCarreTerm` qui utilise une fonction auxiliaire et un accumulateur.

```
-- réponse
sommeCarreTerm :: [Integer] -> Integer
sommeCarreTerm ls = aux ls 0
  where aux [] res = res
        aux (x:xs) res = aux xs (res+x*x)
```

3. Définir une fonction `substituer` qui prend en entrée une chaîne de caractères, remplace chaque voyelle `e` par une étoile `'*`', puis renvoie la chaîne ainsi obtenue. La définition doit utiliser une fonction d'ordre supérieur qui prend en entrée une fonction anonyme.

Exemple : `substituer "une informaticienne" ==> "un* informatici*nn*"`

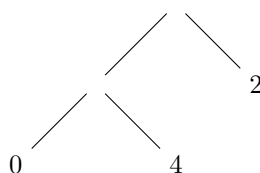
```
-- réponse
substituer :: String -> String
substituer mot = map (\x -> if x=='e' then '*' else x) mot
```

Exercice 3 : (7 points)

On considère un arbre de codage binaire dont les nœuds internes sont non-étiquetés et dont les feuilles sont étiquetées par une valeur de type polymorphe.

Exemple : `a1 = (Bin (Bin (Leaf 0) (Leaf 4)) (Leaf 2))`

dont la représentation graphique est donnée par :



1. Définir la structure **Btree** d'arbre binaire décrite ci-dessus qui prend pour les feuilles un nom de valeurs **Leaf** et pour les nœuds internes un nom de valeurs **Bin**. Ce type doit être affichable.

On encode une liste d'éléments (de type polymorphe) de la manière suivante : chaque étiquette **x** de la liste est remplacée par la description de la succession de Direction – **Left** ou **Right** – à parcourir pour aller de la racine de l'arbre à la feuille contenant l'étiquette **x**. Le codage de la liste correspond à la concaténation des codages de ses éléments. Bien entendu, toutes les étiquettes de la liste sont des étiquettes de l'arbre.

Pour l'arbre **a1** d'entiers ci-dessus, pour aller de la racine à la valeur 4 on doit aller à gauche puis à droite. Ainsi, l'encodage de la liste **[4, 0]** avec ce graphe donne **[Left, Right, Left, Left]**.

2. Définir :

- un type binaire **Direction** qui peut prendre les valeurs : **Left** pour aller à gauche et **Right** pour aller à droite,

- le type **Code** qui est constitué d'une liste de valeurs de type **Direction**,

Ces types doivent être affichables.

-- réponse

```
data Direction = Left | Right deriving Show
type Code = [Direction]
data Btree a = Leaf a | Bin (Btree a) (Btree a) deriving (Show)
```

3. Définir la fonction **decoder** qui étant donnée un arbre de codage et un code retourne la liste décodée.

Exemple : si **c1 = [Right, Left, Left, Right, Left, Right]** alors

decoder a1 c1 ==> [2, 0, 2, 4]

-- réponse

```
decodeElement :: (Btree a) -> Code -> (a, Code)
decodeElement (Leaf x) xs = (x, xs)
decodeElement (Bin g _) (Left:xs) = decodeElement g xs
decodeElement (Bin _ d) (Right:xs) = decodeElement d xs

decoder :: (Btree a) -> Code -> [a]
decoder (Leaf x) [] = [x]
decoder tree [] = []
decoder tree code = x:decoder tree resteCode
  where (x, resteCode) = decodeElement tree code
```

Département mathématiques et informatique
UFR des Sciences

**Licence informatique - 2ème année
(YSINL4B1) Programmation Fonctionnelle
1 heure et 30 minutes**

Vous devez remettre ce sujet à votre surveillant à la fin du TP noté.

Les notes prises pendant les séances CM et TP en version papier sont autorisées, ainsi que les TP préparés et le support du cours dans vos répertoires personnels sur votre compte informatique de l'université de Caen Normandie ou sur la page ecampus du cours¹. L'accès à internet, les ordinateurs portables, téléphones portables, montres, clés usb et plus généralement tout matériel électronique sont interdits.

Le TP noté est à faire **individuellement** et **exclusivement sur une machine de l'université de Caen Normandie**. Toute communication avec une autre personne ou avec une intelligence artificielle est interdite et sera passible d'un conseil de discipline. Les sites consultés sur internet sont enregistrés pour chaque session pendant toute la durée du TP noté et feront l'objet d'une vérification.

Consignes :

- Créez un fichier texte `NomPrenom_NumEtu_GroupeTD.hs` où `NomPrenom` est votre nom et votre prénom, `NumEtu` est votre numéro d'étudiant et `GroupeTD` est votre groupe de TD.
- Écrivez (en commentaire) au début du fichier que vous rendez : votre nom, votre prénom, votre numéro d'étudiant et votre groupe TD.
- Écrivez dans le fichier que vous rendez vos réponses aux questions du TP *en indiquant à chaque fois le numéro de la question traitée*.
- A la fin du TP, téléversez (i.e. upload) votre fichier sous ecampus au dépôt **Dépôt du TP noté INFO** disponible en haut de la page du cours sur ecampus :
 - cet envoi doit être fait impérativement AVANT 15H30
 - conservez une COPIE de votre fichier et vérifiez bien que le fichier envoyé est le BON.
 - les envois par mail ne sont pas acceptés.

Toutes les fonctions définies pour les questions suivantes **doivent être signées**.

Exercice 1 : (6 points)

1. Définir une fonction qui prend en entrée les longueurs des côtés d'un triangle et retourne le type de triangle formé (équilatéral, isocèle ou scalène). On définit un triangle *équilatéral* lorsque ses trois côtés ont la même longueur, un triangle *isocèle* lorsque deux de ses côtés ont la même longueur et le troisième côté a une longueur différente, triangle *scalène* lorsque ses trois côtés ont des longueurs différentes.

Exemple : `typeTriangle 3 3 3 => "Équilatéral"`, `typeTriangle 3 3 4 => "Isocèle"`,
`typeTriangle 3 4 5 => "Scalène"`

1. <https://ecampus.unicaen.fr/course/view.php?id=53232>

```
-- réponse
typeTriangle :: Int -> Int -> Int -> String
typeTriangle a b c
  | a == b && b == c = "Équilatéral"
  | a == b || a == c || b == c = "Isocèle"
  | otherwise = "Scalène"
```

2. Définir une fonction `calculSuite` qui retourne le n-ième de la suite définie par :

$$\begin{cases} u_0 &= 10 \\ u_{n+1} &= \frac{2}{3}u_n + 1 \end{cases}$$

Exemple : `calculSuite 7 ==> 3.4096937`, `calculSuite 3 ==> 5.0740743`

```
-- réponse
calculSuite :: Float -> Float
calculSuite n
  | n == 0 = 10
  | otherwise = 1 + 2 * ( calculSuite ( n - 1 ) ) / 3
```

3. Définir à l'aide d'une ZF-expression une fonction `sommeSuite` qui retourne la somme des (n+1) premiers termes de la suite donnée dans la question précédente.

$$\{ S_n = \sum_0^n u_n$$

Par conséquent, on utilisera la fonction `calculSuite`. Vous pouvez utiliser une fonction standard d'Haskell pour faire la somme sur une liste.

Exemple : `sommeSuite 7 ==> 44.18061`, `sommeSuite 5 ==> 37.156376`

```
-- réponse
sommeSuite :: Float -> Float
sommeSuite n = sum [ ( calculSuite x ) | x <- [0..n] ]
```

4. Définir une fonction qui prend en paramètres une liste d'entiers `xs` et un seuil en entiers `x` et retourne le produit des éléments de cette liste qui sont supérieurs strictement à au seuil donné `x`. Votre définition doit utiliser une fonction d'ordre supérieur telle que `map`, `filter`,... Vous pouvez utiliser une fonction standard d'Haskell pour faire le produit sur une liste.

Exemple : `produitSupSeuil [1,2,3,4] 2 ==> 12`

```
-- réponse
produitSupSeuil :: [Int] -> Int -> Int
produitSupSeuil xs seuil = product (filter (> seuil) xs)
```

Exercice 2 : (7 points)

1. Le code de César est une méthode de chiffrement très simple utilisée par Jules César dans ses correspondances secrètes. Le texte chiffré s'obtient en remplaçant chaque lettre du texte clair original par une lettre à distance fixe. Définir la fonction `decaler` qui prend en paramètres la distance ainsi que la lettre à chiffrer et retourne le caractère correspondant en appliquant la distance donnée. Nous considérons que des lettres dans l'alphabet, écrites en majuscules.

Exemple : `decaler 3 'A' ==> 'D'`, `decaler 3 'Z' ==> 'C'`, `decaler 5 'H' ==> 'M'`

```
--réponse
decaler 0 x = x
decaler d x
  | d >= 26 = error "clé non valide"
  | otherwise = let suivant 'Z' = 'A'
                --          suivant 'z' = 'a'
                suivant x = succ x
                in decaler (d-1) (suivant x)
```

2. On envisage la fonction `inverser` suivante qui prend en paramètre une liste d'éléments et retourne une liste ayant les mêmes éléments mais dans l'ordre inverse des positions initiales :

```
inverser :: [a] -> [a]
inverser [] = []
inverser (x:xs) = inverser xs ++ [x]

-- inverser "trop" ==> "port"
-- inverser [4,2,7,9] ==> [9,7,2,4]
```

Cette fonction est récursive non-terminale. Redéfinir cette fonction en une fonction terminale `inverserTerm` qui utilise une fonction auxiliaire et un accumulateur.

```
-- réponse
inverserTerm :: [a] -> [a]
inverserTerm xs = inverserAux xs []
  where inverserAux [] acc = acc
        inverserAux (x:xs) acc = inverserAux xs (x:acc)
```

3. L'objectif est de résoudre l'équation de second degré dans le cas où les solutions sont réelles. L'équation $ax^2 + bx + c = 0$ admet une solution unique réelle si $(\Delta = b^2 - 4ac)$ est égal à zéro. Elle admet deux solutions réelles si $\Delta > 0$. La solution double est donnée par $x = -\frac{b}{2a}$ et les deux solutions $x_1 = \frac{-b + \sqrt{\Delta}}{2a}$ et $x_2 = \frac{-b - \sqrt{\Delta}}{2a}$. Si $\Delta < 0$, on prend que l'équation n'a pas de solutions réelles. Nous définissons le type suivant pour résoudre les différentes équations :

```
type Equation = (Float,Float,Float) -- On représente le polynôme de l'équation
                                     -- par triplet de coefficients.
data Solution = SolUnique Float | SolDouble Float Float | Rien -- Trois valeurs possibles
deriving Show -- de solutions
```

Définir une fonction `resoudreEquation` qui renvoie les solutions dans les cas réels.

Exemple : `resoudreEquation (5,2,-1) ==> (-0.68989795,0.28989798)`, `resoudreEquation (5,2,1) ==> Rien`

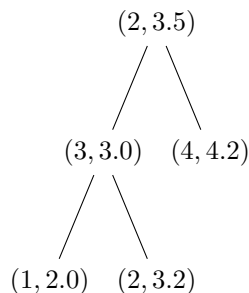
```
--réponse
resoudreEquation :: Equation -> Solution
resoudreEquation (a,b,c)
  | (delta ) == 0 = SolUnique (- b / (2 * a ))
  | (delta ) > 0 = SolDouble ((- b - sqrt delta ) / (2 * a )) ((- b + sqrt delta ) / (2 * a ))
  | (delta ) < 0 = Rien
  where
    delta = dp (a,b,c)
    dp (a,b,c) = b * b - (4 * a * c)
```

Exercice 3 : (7 points)

Un agriculteur fait une étude statistique sur une population de poulets d'après leur âge (en nombre de mois) et leur poids (en kg). Les données, appelées mesures et de la forme (age, poids), sont stockées dans un arbre binaire de recherche dont la clé de comparaison est le poids : pour chaque nœud `no` dont la mesure est `(a, p)`, les poids des mesures contenues dans le sous-arbre fils gauche de `no` est inférieure ou égale à `p` et ceux des mesures du sous-arbre fils droit strictement supérieure à `p`.

Exemple : `a = Revele (2, 3.5) (Releve (3,3) (Releve (1,2) Vide Vide) (Releve (2,3.2) Vide Vide) ) (Releve (4,4.2) Vide Vide)`

dont la représentation graphique est donnée par :



1. Définir :

- un type `Mesure` qui est constitué d'un tuple ayant le nombre de mois et le poids,
- la structure `ArbreMesure` d'arbre binaire de recherche qui prend pour tous les nœuds un nom de valeurs `Releve` et `Vide` pour un arbre vide.

Ces types doivent être affichables.

--réponse

```
type Measure = (Integer, Float)
data ArbreMesure = Vide | Releve Measure ArbreMesure ArbreMesure deriving Show
```

2. Définir une fonction `poidsCommun` qui, étant donnée un arbre de mesures ainsi que deux poids `p1` et `p2`, retourne le poids commun des deux poids `p1` et `p2`. Un poids commun entre deux poids dans l'arbre est le poids situé dans le nœud ancêtre le plus proche reliant les deux mesures. Pour l'arbre `a` ci-dessus, nous avons

Exemple : `poidsCommun 2.0 4.2 a ==> 3.5`, `poidsCommun 2.0 3.2 a ==> 3.0`

--réponse

```
poidsCommun :: Float -> Float -> ArbreMesure -> Float
poidsCommun p1 p2 (Releve (mois,poids) g d)
  | p1 < poids && p2 < poids = poidsCommun p1 p2 g
  | p1 > poids && p2 > poids = poidsCommun p1 p2 d
  | otherwise = poids
poidsCommun _ _ _ = -1
```

3. Définir une fonction `reOrganiserMois` qui, étant donnée un arbre de mesures organiser selon les poids, retourne l'arbre binaire de recherche dont la clé de comparaison est le mois.

Exemple : `reOrganiserMois a ==> Revele (3,3.0) (Releve (2,3.5) (Releve (1,2.0) Vide Vide) (Releve (2,3.2) Vide Vide)) (Releve (4,4.2) Vide Vide)`

--réponse

```
reOrganiserMois :: ArbreMesure -> ArbreMesure
reOrganiserMois (Releve (m,p) (Releve (mg,pg) gg gd) (Releve (md,pd) dg dd))
  | m < mg = Releve (mg,pg) (reOrganiserMois (Releve (m,p) gg gd)) (Releve (md,pd) dg dd)
  | m > md = Releve (md,pd) (Releve (mg,pg) gg gd) (reOrganiserMois (Releve (m,p) dg dd))
```

```
| otherwise = Releve (m,p) (reOrganiserMois (Releve (mg,pg) gg gd)) (reOrganiserMois (Releve (md,pd) dg dd))  
reOrganiserMois x = x -- cas de base
```
